

SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: DESIGN AND ANALYSIS OF ALGORITHMS
COURSE CODE: CSA0606

COURSE OUTCOME COVERED

CO2: Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)

Assessment Tool Weightage: 40%

QUESTIONS: 10

Total Marks: 100

Annexure A

ANALYTICAL PROBLEM SOLVING

Code of Conduct:

*I **I. Guru Hemanth Reddy (192311222)**(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: Hemanth Reddy

SCENARIO BASED ASSESSMENT: RIDE-HAILING CLOSEST PAIR ANALYSIS

1 1. Introduction to the Scenario

In the era of on-demand transportation, ride-hailing services (e.g., Uber, Lyft, Grab) have revolutionized urban mobility. A critical operational component of these platforms is the matching engine, which connects a passenger requesting a ride with an available driver. The core objective of this matching engine is to minimize the passenger's wait time and the driver's idle travel distance. In this specific scenario, the ride-hailing service utilizes a **brute-force closest pair comparison** to identify and assign the nearest available driver to a waiting passenger.

2 2. Scenario Analysis and Problem Statement

Problem Statement: Given a single query point (the passenger's location) and a set of n dynamic points (the locations of all currently available drivers), find the driver whose geographic distance to the passenger is the absolute minimum using an exhaustive brute-force search.

Scenario Analysis: When a user opens the app and taps "Request Ride", their smartphone transmits their current GPS coordinates (latitude, longitude) to the central server. The server must then look at its registry of all online, unoccupied drivers. Using the brute-force approach, the server calculates the exact straight-line (or Haversine) distance between the passenger and *every single driver* in the database. The driver corresponding to the smallest calculated distance is dispatched. While mathematically flawless in guaranteeing the absolute closest driver is found, this exhaustive approach raises significant questions regarding computational efficiency and scalability as the user base expands.

3 3. User Requirements and System Objectives

3.0.1 Functional Requirements:

- **Coordinate Acquisition:** The system must continuously capture real-time GPS coordinates of active drivers and instantaneous coordinates of requesting passengers.
- **Distance Computation:** The system must implement a reliable mathematical formula (Euclidean distance for projected grids or Haversine for spherical Earth) to compute the distance between two coordinate pairs.

- **Optimal Assignment:** The system must successfully identify the absolute minimum distance and dispatch the corresponding driver.

3.0.2 Non-Functional Requirements:

- **Low Latency (Response Time):** The matching algorithm must execute in real-time, ideally returning a driver match in under 3 seconds to maintain a positive user experience.
- **High Accuracy:** The distance logic must be precise to avoid assigning drivers who are geographically distant, which wastes fuel and frustrates passengers.
- **High Availability:** The server architecture must remain operational despite concurrent ride requests.

3.0.3 System Objectives:

- To minimize the Estimated Time of Arrival (ETA) for passengers.
- To maximize fleet utilization by keeping driver dispatch distances short.
- To evaluate the computational limits of the brute-force architecture and identify transition points for upgrading to advanced spatial indexing.

4 4. Brute-Force Closest Pair Algorithm Explanation

The brute-force closest pair algorithm works on the fundamental principle of exhaustive search. It leaves no possibility unchecked. 1. **Initialization:** The algorithm starts by assuming the minimum distance is infinitely large ($\text{min_distance} = \infty$). 2. **Iteration:** It loops sequentially through the list of all available drivers. 3. **Calculation:** For every single driver, it computes the exact geographic distance to the passenger. 4. **Comparison and Update:** If the newly calculated distance is smaller than the current min_distance , the algorithm updates min_distance with this new value and records the driver's ID as the best candidate. 5. **Termination:** After the loop has visited every single driver in the dataset exactly once, it terminates. The recorded driver is mathematically guaranteed to be the nearest one.

5 5. Algorithm Pseudocode

```
Algorithm FindNearestDriver(Passenger, DriversArray, n)
// Input: Passenger object with coordinates (x, y)
//       DriversArray containing n available drivers with coordinates (x, y)
// Output: The DriverID of the nearest driver

min_distance = INFINITY
nearest_driver_id = NULL
```

```

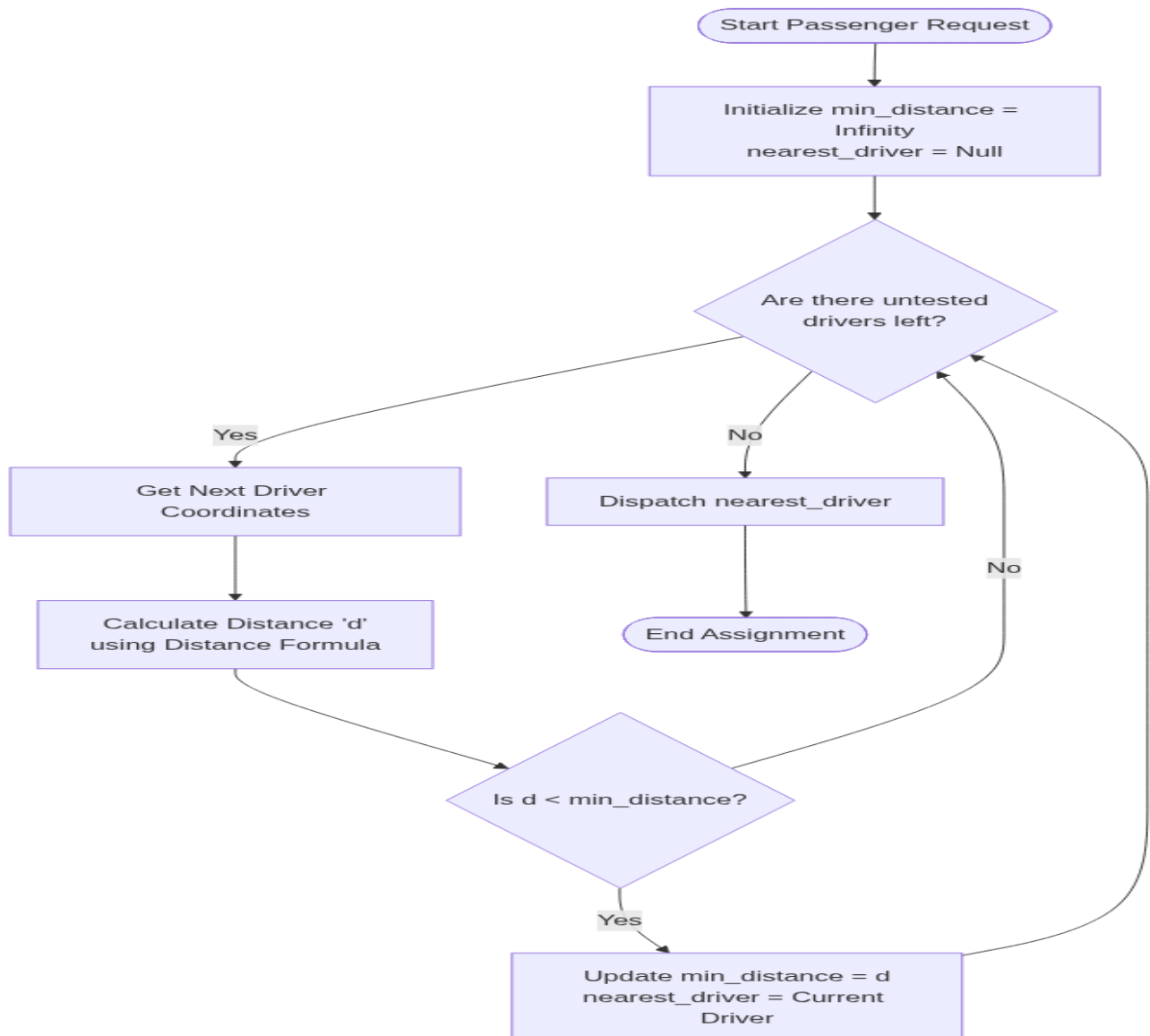
For i = 0 to n - 1 do
    // Calculate Euclidean distance (or Haversine)
    dx = DriversArray[i].x - Passenger.x
    dy = DriversArray[i].y - Passenger.y
    distance = sqrt(dx^2 + dy^2)

    // Compare and update minimum
    If distance < min_distance then
        min_distance = distance
        nearest_driver_id = DriversArray[i].id
    End If
End For

Return nearest_driver_id

```

6 6. Flowchart (Mermaid)



7 7. Dry Run with Sample Input

Sample Input: * Passenger Location: $P(3,4)$ * Available Drivers: * Driver A: $D_1(0,0)$ * Driver B: $D_2(3,1)$ * Driver C: $D_3(7,7)$ * Driver D: $D_4(4,5)$

Step-by-Step Execution: * **Initialization:** $\text{min_distance} = \infty$, $\text{nearest_driver} = \text{None}$ * **Iteration 1 (Driver A at 0,0):** * $d = \sqrt{(0-3)^2 + (0-4)^2} = \sqrt{9+16} = \sqrt{25} = 5.0$ * $5.0 < \infty \Rightarrow \text{min_distance} = 5.0$, $\text{nearest_driver} = \text{Driver A}$. * **Iteration 2 (Driver B at 3,1):** * $d = \sqrt{(3-3)^2 + (1-4)^2} = \sqrt{0+9} = \sqrt{9} = 3.0$ * $3.0 < 5.0 \Rightarrow \text{min_distance} = 3.0$, $\text{nearest_driver} = \text{Driver B}$. * **Iteration 3 (Driver C at 7,7):** * $d = \sqrt{(7-3)^2 + (7-4)^2} = \sqrt{16+9} = \sqrt{25} = 5.0$ * $5.0 \not< 3.0 \Rightarrow$ No update. * **Iteration 4 (Driver D at 4,5):** * $d = \sqrt{(4-3)^2 + (5-4)^2} = \sqrt{1+1} = \sqrt{2} \approx 1.41$ * $1.41 < 3.0 \Rightarrow \text{min_distance} = 1.41$, $\text{nearest_driver} = \text{Driver D}$. * **Result:** The loop ends. The passenger is assigned to **Driver D** at a distance of 1.41 units.

8 8. Time and Space Complexity Derivation

8.0.1 Time Complexity:

- **Best Case ($O(n)$):** Even if the very first driver checked is physically parked exactly where the passenger is standing (distance = 0), the brute-force algorithm does not know this is the absolute minimum until it checks the rest of the array. Therefore, it still loops n times.
- **Average Case ($O(n)$):** For an arbitrary distribution of drivers, the algorithm always evaluates the distance formula exactly n times.
- **Worst Case ($O(n)$):** The algorithm must iterate through all n drivers to confirm the shortest distance, requiring n square root and arithmetic operations.
- **Overall Time Complexity:** $\theta(n)$ per passenger request. If m passengers request rides simultaneously, the total system complexity becomes $O(m \times n)$.

8.0.2 Space Complexity:

- **Space Complexity ($O(1)$ auxiliary):** The algorithm only requires a few constant variables (min_distance , nearest_driver_id , and temporary calculation variables) regardless of the size of n . Assuming the driver list is already held in the database/memory, the auxiliary space complexity is $O(1)$.

9 9. Algorithm Correctness

The algorithm is mathematically sound and unconditionally correct. It relies on the transitive property of inequalities. By evaluating the distance to *every* possible candidate in the domain, it is impossible for a closer driver to be overlooked. If a driver D_{closer} existed, they would eventually be processed by the loop, their distance would evaluate as

strictly less than the current `min_distance`, and they would replace the incumbent candidate. Hence, the brute-force method perfectly guarantees finding the absolute nearest driver without heuristic estimation errors.

10 10. Scalability Issues as the Number of Users Increases

While $O(n)$ is efficient for small datasets, it becomes a critical bottleneck in hyper-growth scenarios. * **Compute Bottleneck:** If a city like New York has 50,000 active drivers ($n = 50,000$), a single ride request requires 50,000 distance calculations. If 1,000 passengers ($m = 1,000$) open the app during rush hour, the server must perform $50,000 \times 1,000 = 50,000,000$ geometric distance calculations per second. This leads to massive CPU throttling. * **Geographic Waste:** A brute-force algorithm searches globally. If a passenger requests a ride in downtown Manhattan, the algorithm wastes thousands of CPU cycles calculating the exact distance to drivers who are miles away in Brooklyn or New Jersey. * **Response Time Degradation:** As CPU loads spike due to the $O(m \times n)$ complexity, the server response time degrades. The passenger's app hangs on the "Finding Driver" screen, eventually causing timeouts, lost revenue, and user abandonment.

11 11. Suggestion of Better Algorithms and Comparison

To resolve the $O(n)$ bottleneck, ride-hailing platforms utilize **Spatial Indexing**. These data structures segment the 2D world into searchable grids or trees, allowing the algorithm to search only the immediate vicinity of the passenger (turning $O(n)$ into $O(\log n)$ or $O(1)$).

- **Geohashing:** Divides the Earth into a grid of alphanumeric strings. The system only searches for drivers sharing the same Geohash prefix as the passenger.
- **Quad Tree:** A tree data structure where each node represents a bounding box, recursively subdivided into four quadrants. It efficiently filters out large swaths of the map that are far from the passenger.
- **KD-Tree (k-Dimensional Tree):** A binary search tree partitioning space into half-spaces, exceptionally fast for nearest-neighbor searches in static datasets, though balancing it with constantly moving drivers is challenging.
- **R-Tree:** A tree data structure used for spatial access methods, grouping nearby objects into minimum bounding rectangles (MBRs).
- **Grid Indexing:** The map is divided into a fixed 2D array of cells. The passenger queries their cell and immediately adjacent cells, providing $O(1)$ lookup times.

12 12. Comparison Table of Algorithms

Feature	Brute Force	KD-Tree	Quad Tree	Geohashing
---------	-------------	---------	-----------	------------

Feature	Brute Force	KD-Tree	Quad Tree	Geohashing
Search Time Complexity	$O(n)$	$O(\log n)$	$O(\log n)$	$O(1)$ (Prefix match)
Data Structure Complexity	Low (Simple Array)	High (Binary Tree)	High (4-ary Tree)	Medium (String Hash Map)
Handling Moving Drivers	Excellent (No updates needed)	Poor (Requires constant rebalancing)	Moderate (Nodes must be updated)	Excellent (Simple string update)
Memory Overhead	Minimal	High	High	Low
Geographic Filtering	None (Searches whole world)	Yes	Yes	Yes

13 13. Performance Evaluation Table

Metric	Brute Force	Advanced Spatial Indexing (e.g., Geohashing)
Execution Time	Slow for large datasets (linear scaling).	Lightning fast (logarithmic or constant scaling).
Memory Usage	Extremely low (array iteration).	Higher (requires maintaining complex indexes/trees).
Scalability	Poor. Will crash during peak rush hours.	Excellent. Easily handles millions of concurrent global users.
Response Time	Increases linearly as the driver pool grows.	Remains consistently low regardless of fleet size.
Accuracy	100% exact match guaranteed.	100% exact match within the localized search radius.

14 14. Justification: Suitable for Small Datasets but not Large-Scale

Brute force is highly suitable for localized environments—such as a small town taxi service with a fleet of 50 cars, or a college campus shuttle tracker. In these micro-environments, $O(n)$ calculations take fractions of a millisecond. The engineering overhead, maintenance, and database costs required to build and maintain a dynamic Quad Tree for just 50 cars completely outweigh the performance benefits. However, for large-scale systems (like Uber), the brute-force approach fails catastrophically. The dataset of drivers (n) spans millions globally. Calculating the distance between a passenger in London and a driver in Tokyo is mathematically absurd and computationally

irresponsible. Large systems *must* segment their data geographically to survive peak loads.

15 15. Conclusion and Recommendations

Key Findings: The brute-force closest pair algorithm provides a mathematically perfect, easy-to-implement solution for ride-hailing assignments. Its logic is unassailable, guaranteeing the shortest possible dispatch. However, its strictly linear time complexity ($O(n)$) per request renders it computationally nonviable for large-scale, real-world deployment due to severe CPU bottlenecks and lack of geographic pruning.

Recommendations: 1. **For Prototypes/Startups:** Utilize brute force during the MVP (Minimum Viable Product) phase to speed up time-to-market, as the user base will initially be small enough that $O(n)$ is negligible. 2. **For Production Scale:** Implement **Geohashing** combined with an in-memory database like Redis. Redis supports native Geospatial commands (e.g., GEOSEARCH), allowing the backend to instantly query drivers within a specific radius (e.g., 2 kilometers). This eliminates global brute-force checking, dramatically slashes CPU loads, ensures sub-second response times, and seamlessly scales to millions of concurrent global users.